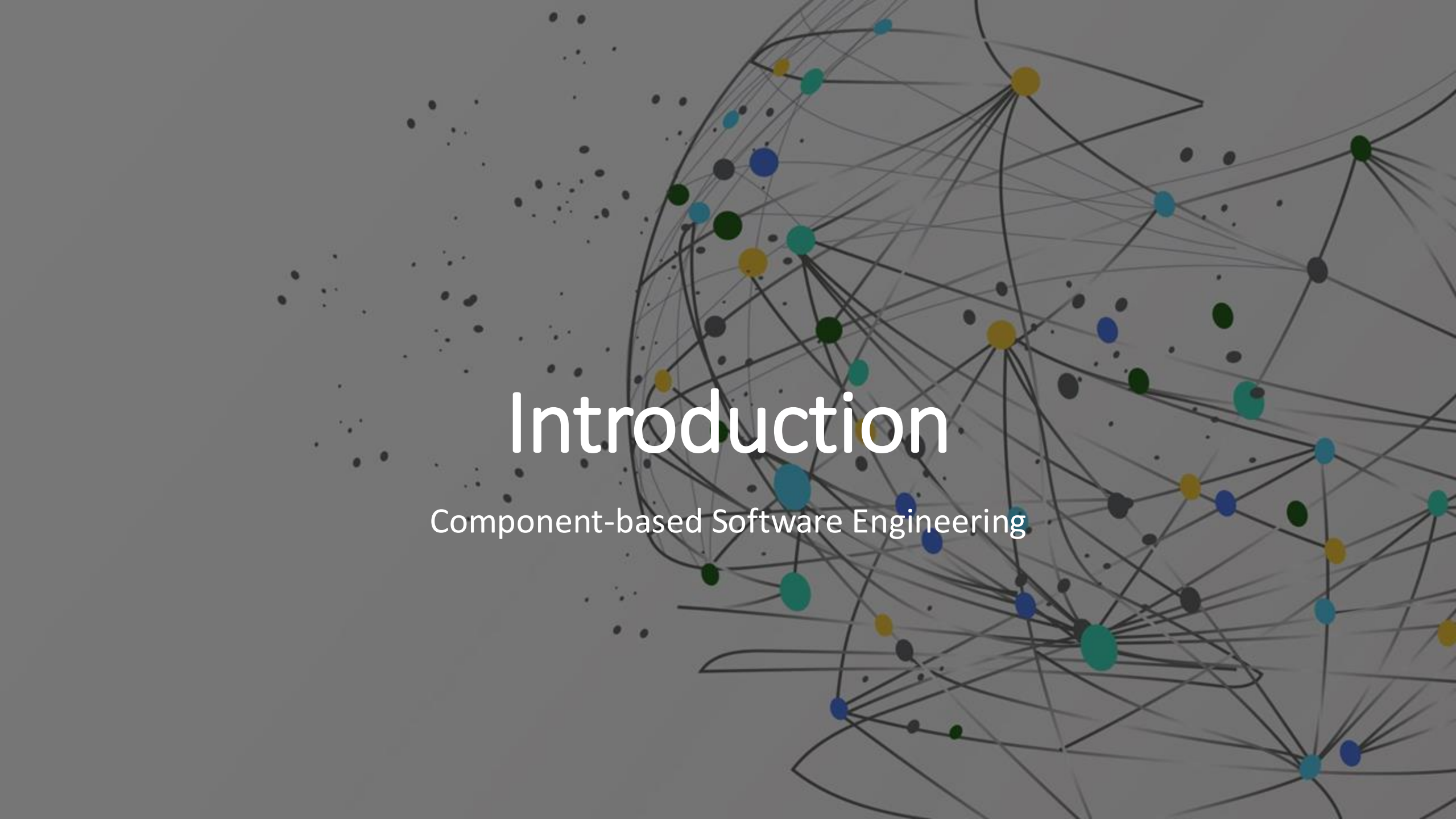


Introduction

Component-based Software Engineering

Staff

- Associate Prof.
  Jan C. Sørensen
- Lectures, instructions and supervision
  jcs@mmmi.sdu.dk

- Teaching Assistant
  Oliver Riis-Vestergaard Møller
  olmoe22@student.sdu.dk

Prerequisites

- Object-oriented programming

Goals

- Understand:
  - Basic principles in CBSE
  - Limitations of 0 0
  - Potential of CBSE
- How to:
  - Identify potential components based on requirements
  - Use Component Models and Java Module System
  - Develop and Test components

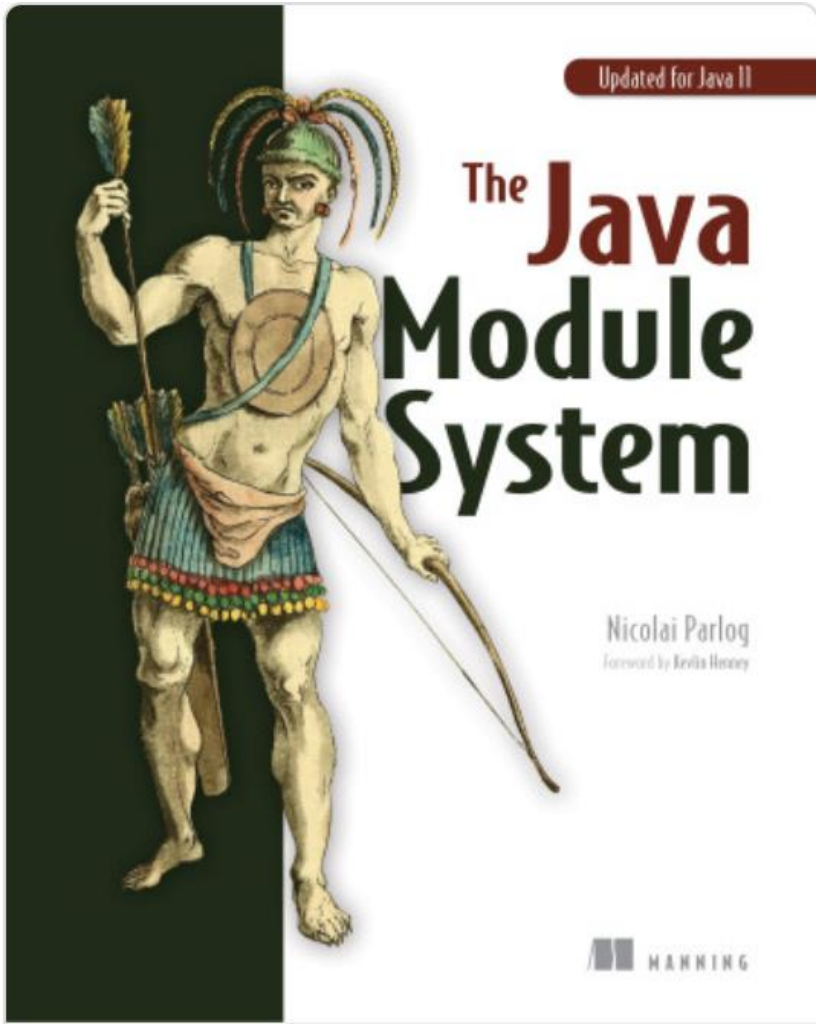
Structure

- Lectures:
  - Ideas, Concepts
  - Examples
- Lab Projects:
  - Programming etc.
  - Component models

Exam

- Mandatory Portfolio
  - Component-based Asteroids
- Written Exam based on portfolio
  - Details later.

Literature

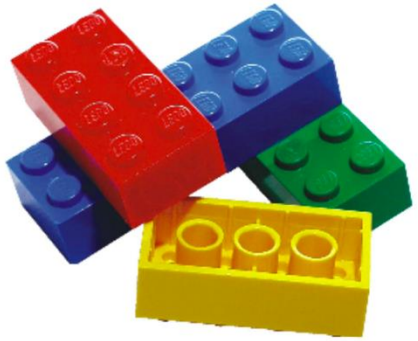


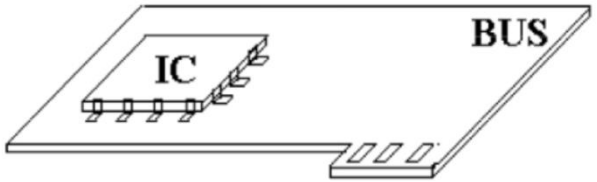
The Java Module System
By Nicolai Parlog
Foreword by Kevlin Henney

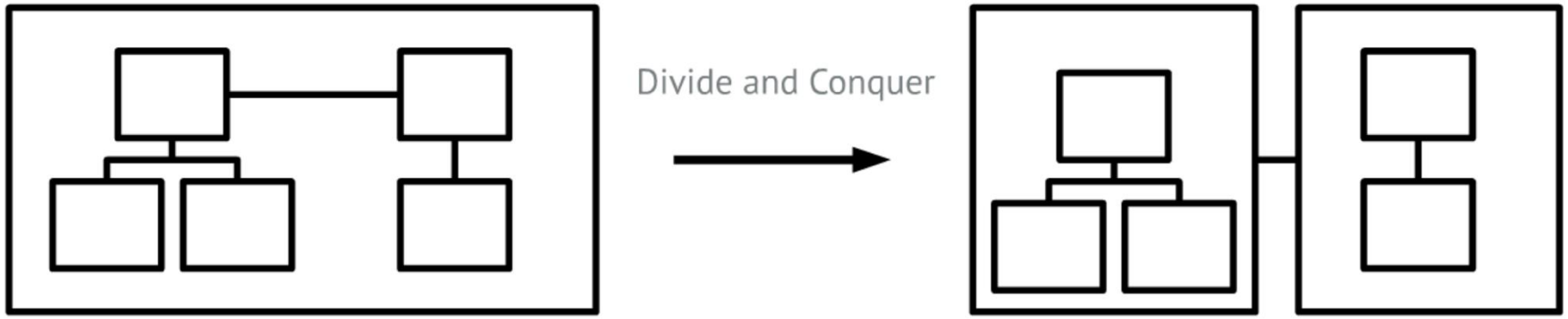
Amazon: https://www.amazon.com/Java-Module-System-Nicolai-Parlog/dp/1617294284

Motivation









Founding Father

"Every ripe industry is based on components, since these allow to manage large systems... Components should be produced in masses and composed to systems afterwards".

Douglas McILroy 1968

Component Vision

Component-based software engineering is concerned with the rapid assembly of systems from components where

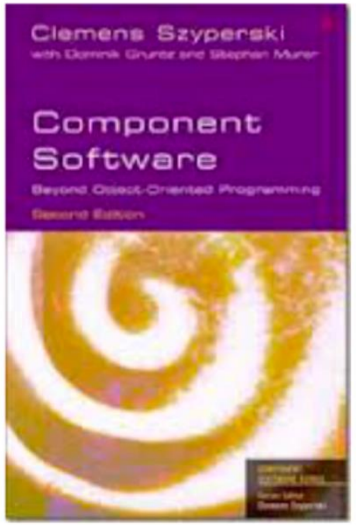
- components and frameworks have certified properties; and
- these certified properties provide the basis for predicting the properties of systems built from components

Definitions



Clemens Alden Szyperski

Dipl.-Ing. (EE/CE) (RWTH Aachen, Germany)
Dr. sc. techn. (CS) (ETH Zurich, Switzerland)
Principal Development Lead in Microsoft Research



Component Software

"A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only...
A software component can be deployed independently and is subject to composition by third parties."

What is CBSE?

- How to create reusable software
- Analysis, Design, Implementation, Testing, Maintenance of components
- Requirements, Specification, Provisioning & Assembly of CBS

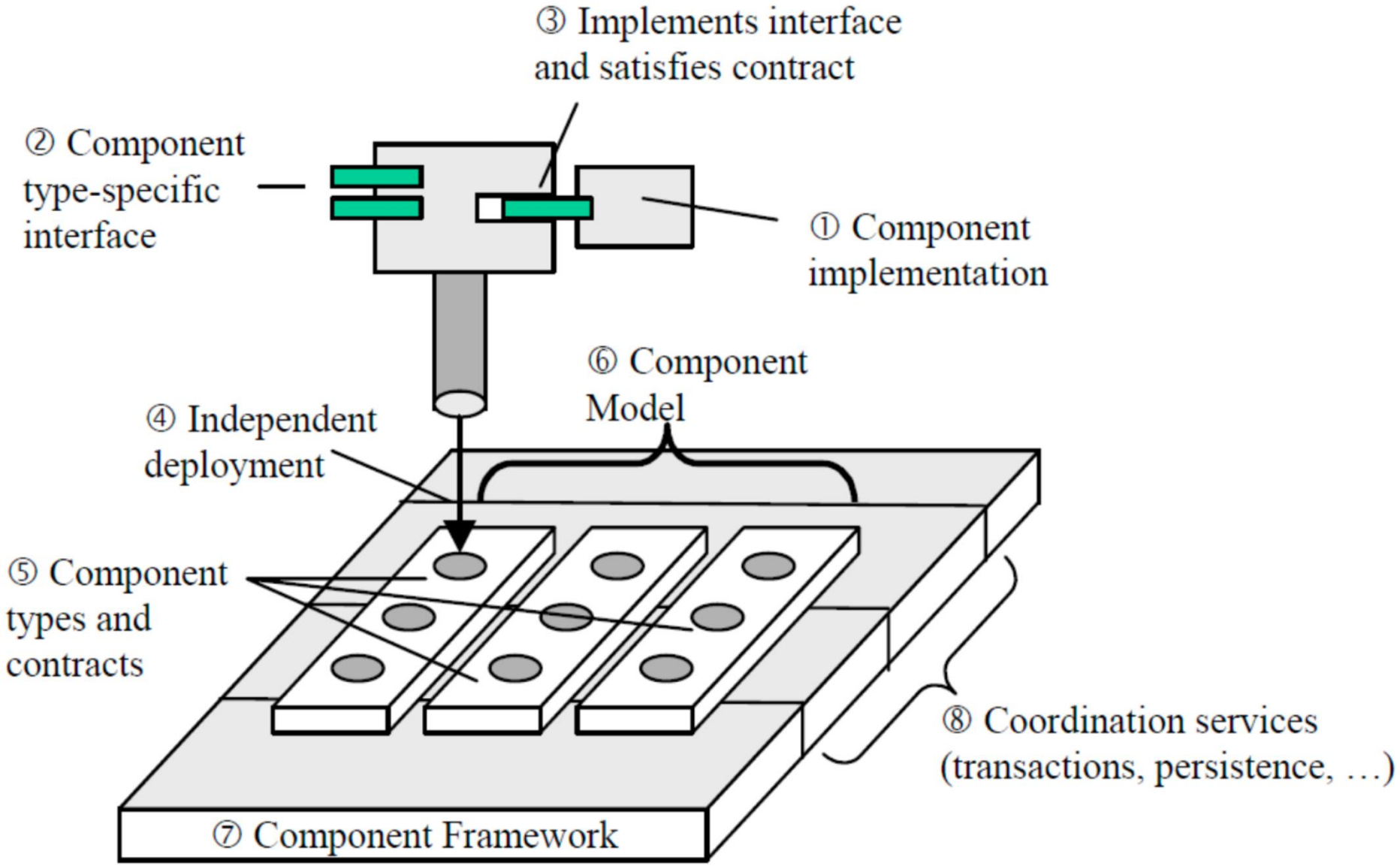
Why CBSE?

- Object technology does not include the notions of independence or late composition
- 00 monolithic applications
- Configuring and integrations normally not possible

Comparison

	Structured Program	OO Program	Component Program
Divide and Conquer	YES	YES	YES
Unification of data and function		YES	YES
Encapsulation		YES	YES
Identity		YES	YES
Contracts			YES

Component Concepts



Component

A Component is:

- an opaque implementation of functionality
- subject to third-party composition
- conformant with a component model

Part of Whole

Composition of an appropriate set of components should yield a complete functional system...



Contract

A component contract specifies a pattern of interaction rooted on that component. The contract specifies the services provided by a component and the obligations of clients and the environment needed by a component to provide these services.

Provided & Required Interfaces

Bolt *provides* **solid cylinder**



Nut *provides* **hollow cylinder**



Bolt *requires* **hollow nut**

Nut *requires* **solid cylinder**

Compatibility



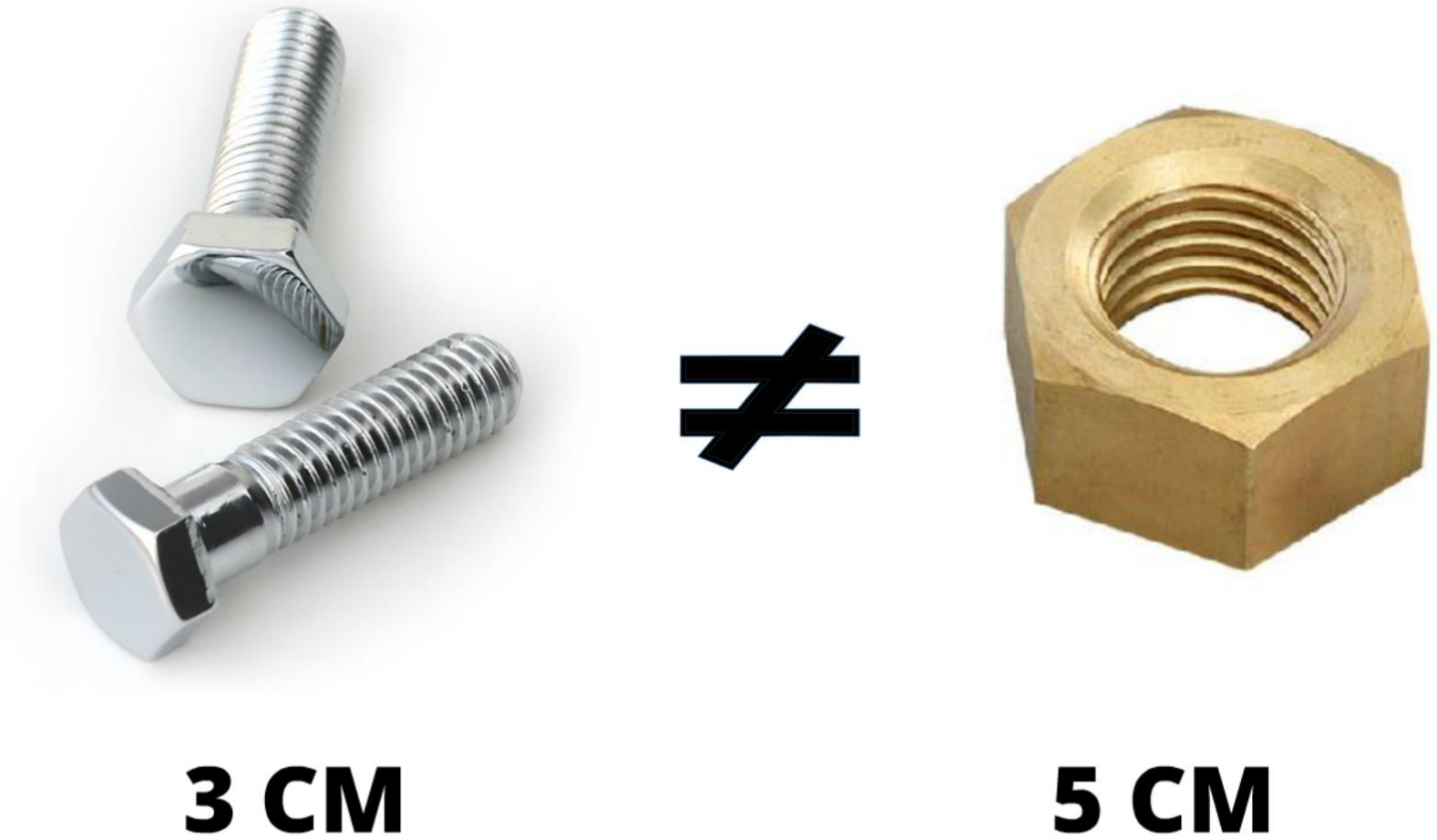
3 CM

≠

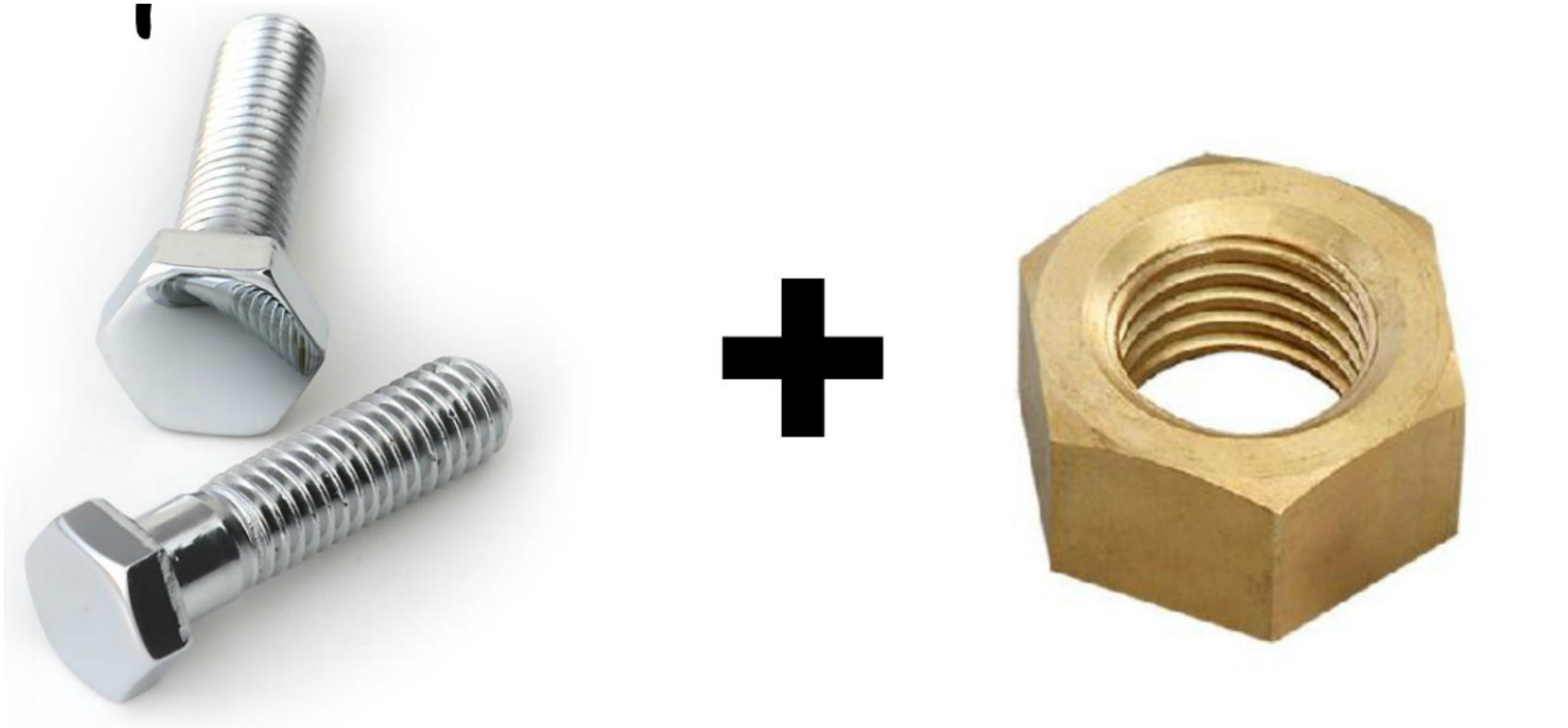


5 CM

Implementation Independence



Producer-Consumer Independence



Company A **Company B**

Interaction Contract

An interaction contract specifies a pattern of interaction among different roles, and the reciprocal obligations of components that fill these roles.

Component Model

A **component model (6)** is the set of component types, their interfaces, and, additionally, a specification of the allowable patterns of interaction among component types

Component Framework

A **component framework (7)** provides a variety of run-time services (8) to support and enforce the **component model**. In many respects component frameworks are like special-purpose operating systems, although they operate at much higher levels of abstraction.

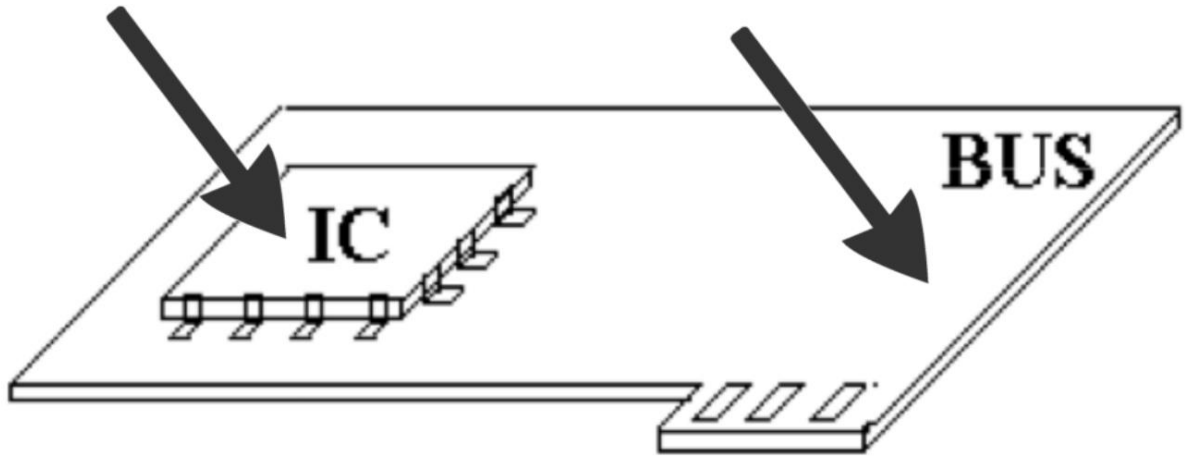


Ecosystem

A common **fabric** or **environment** over which the components may be composed

Component

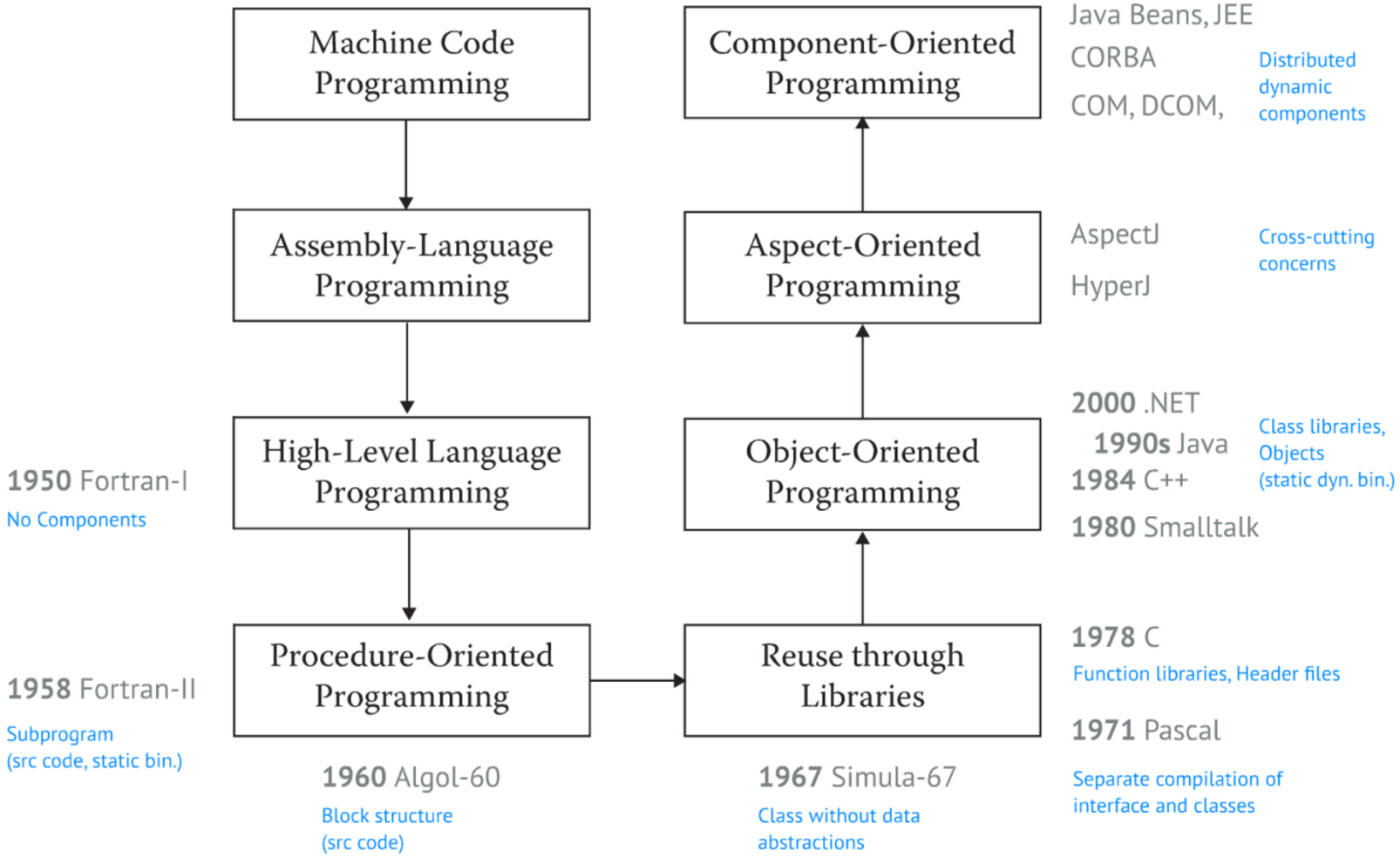
Framework



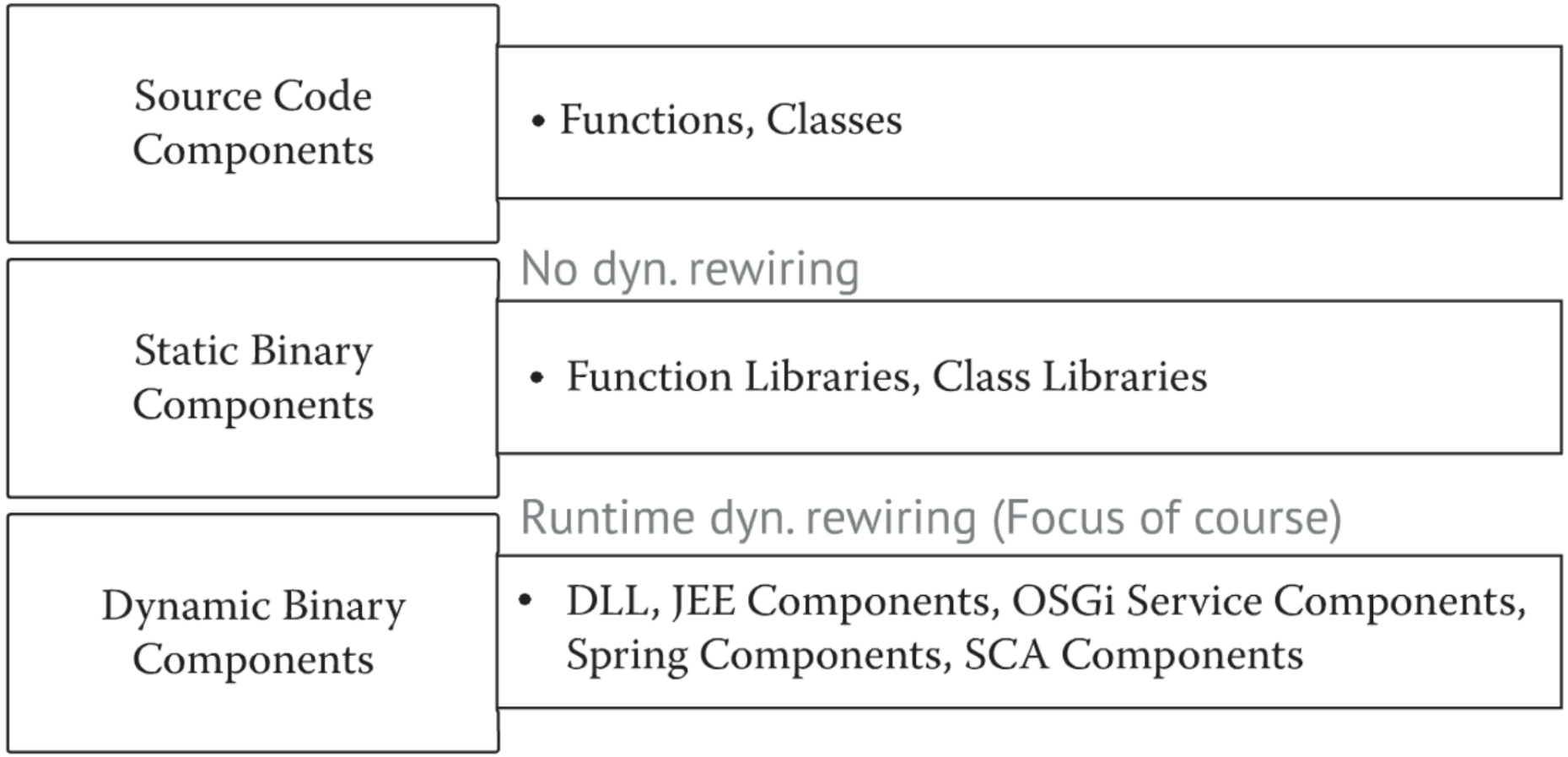
Component Model

Electronics Industries Alliance Standards

History of Programming Languages



Software Components

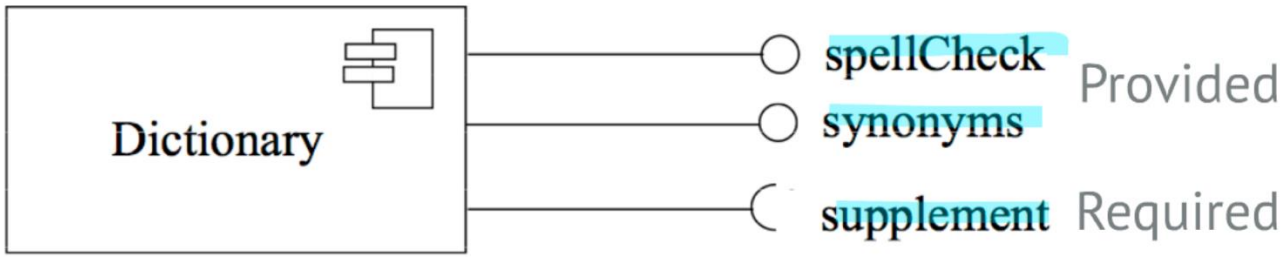


Software Components

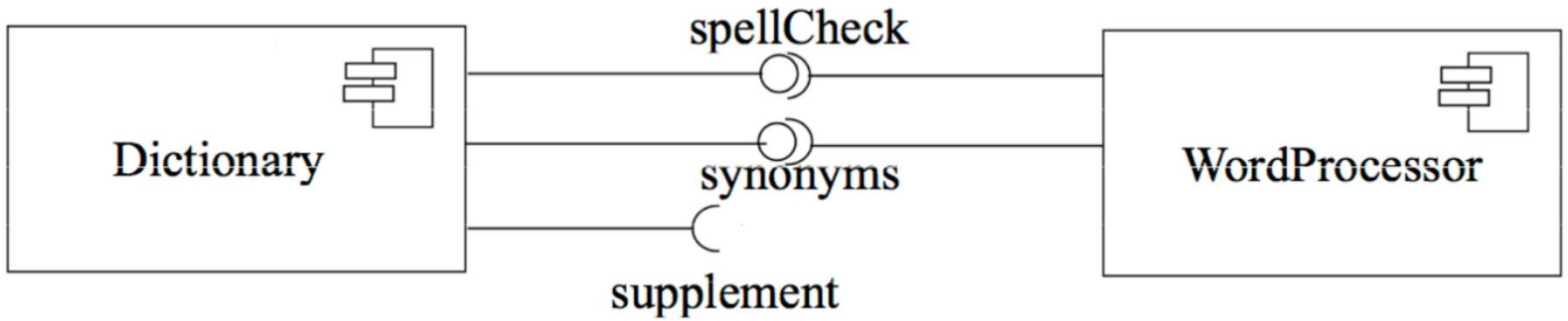
- Specifications
- Interfaces
- Implementation(s)
- Model describing:
  - Composition
  - Storage and distribution
- Run-time framework

UML Components

Component



Wiring

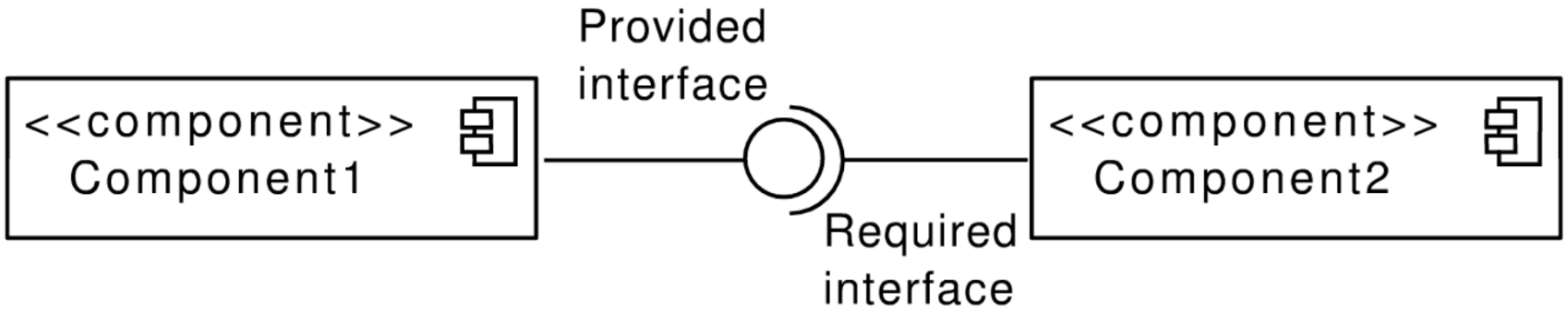


Component VS Object





Specifications



Contract

Postconditions

Versioning required

Syntax

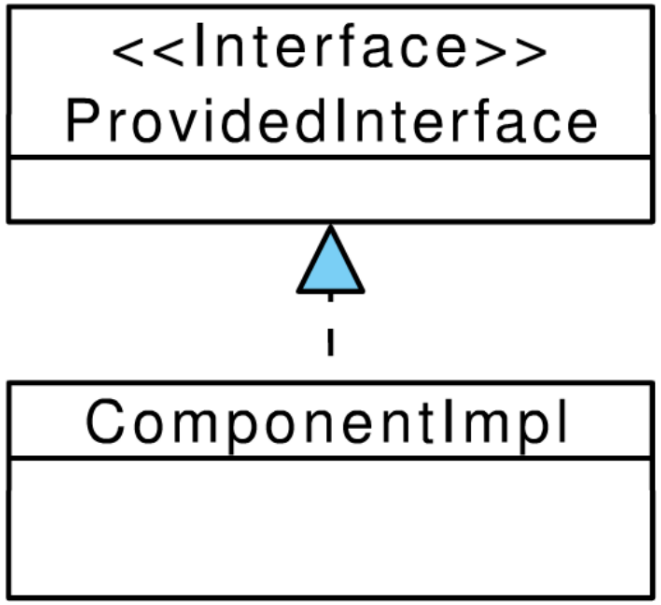
Invariants

Semantics

Preconditions

Quality attributes

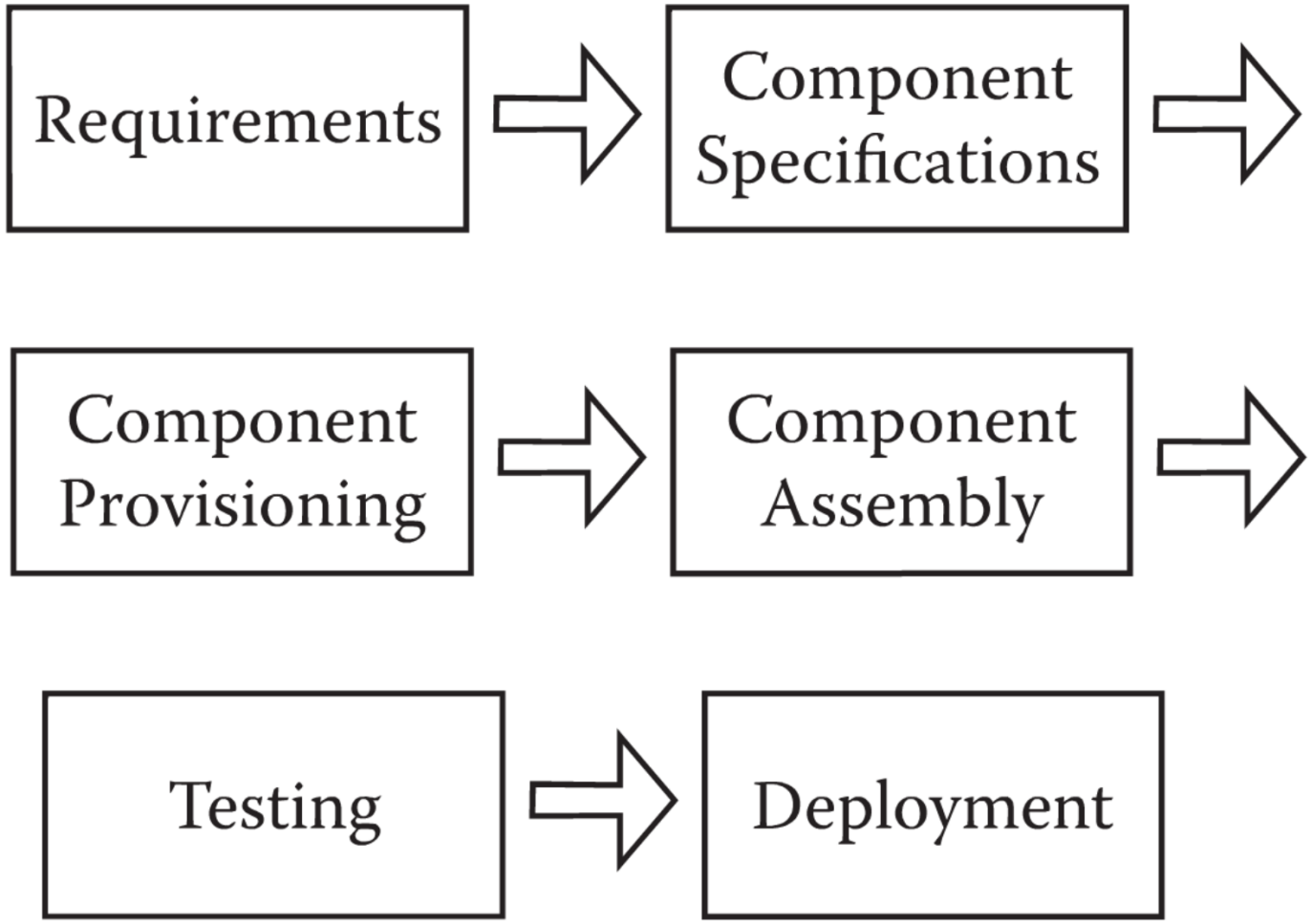
Implementations



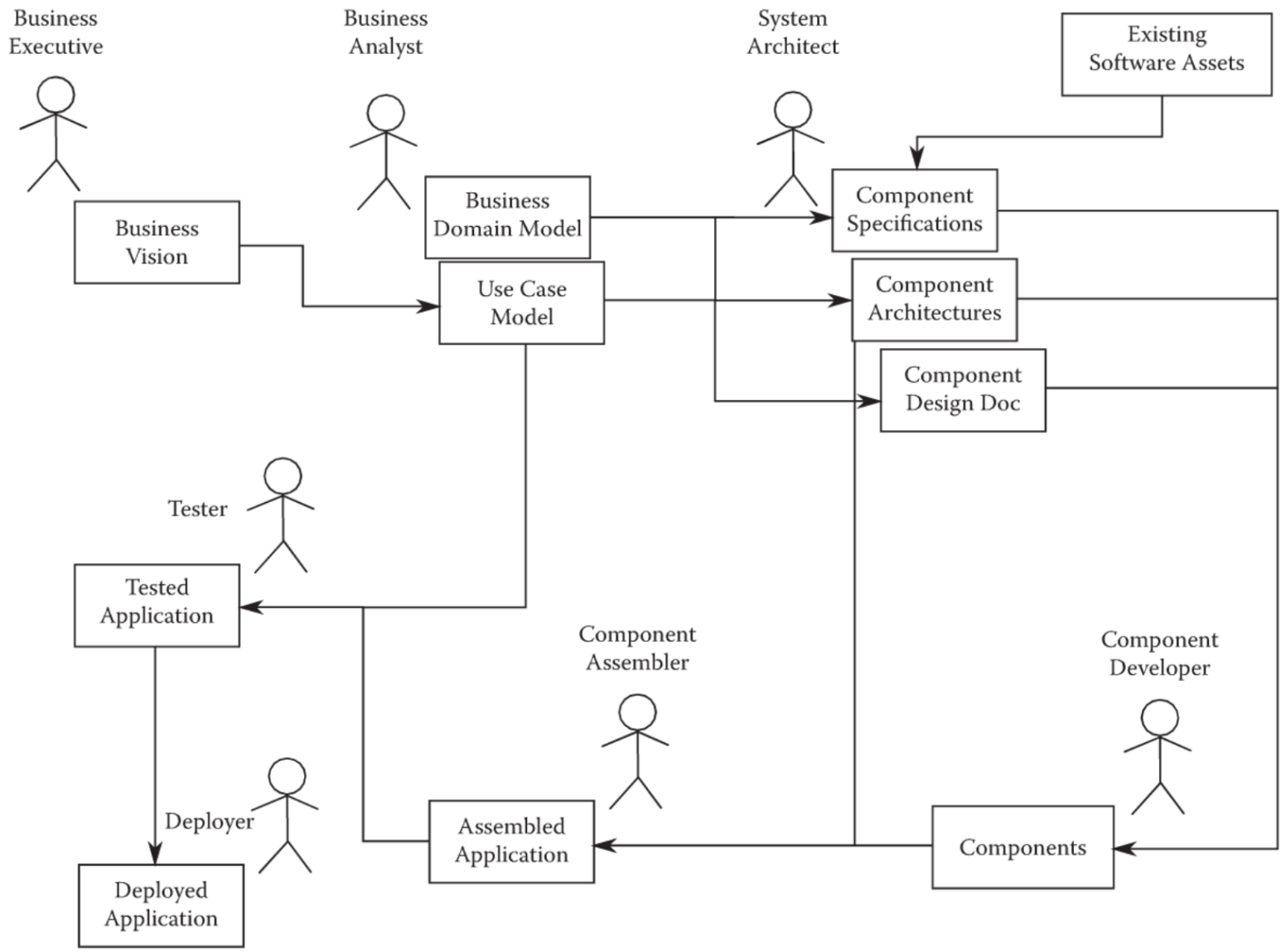
Impl. conform to specs.

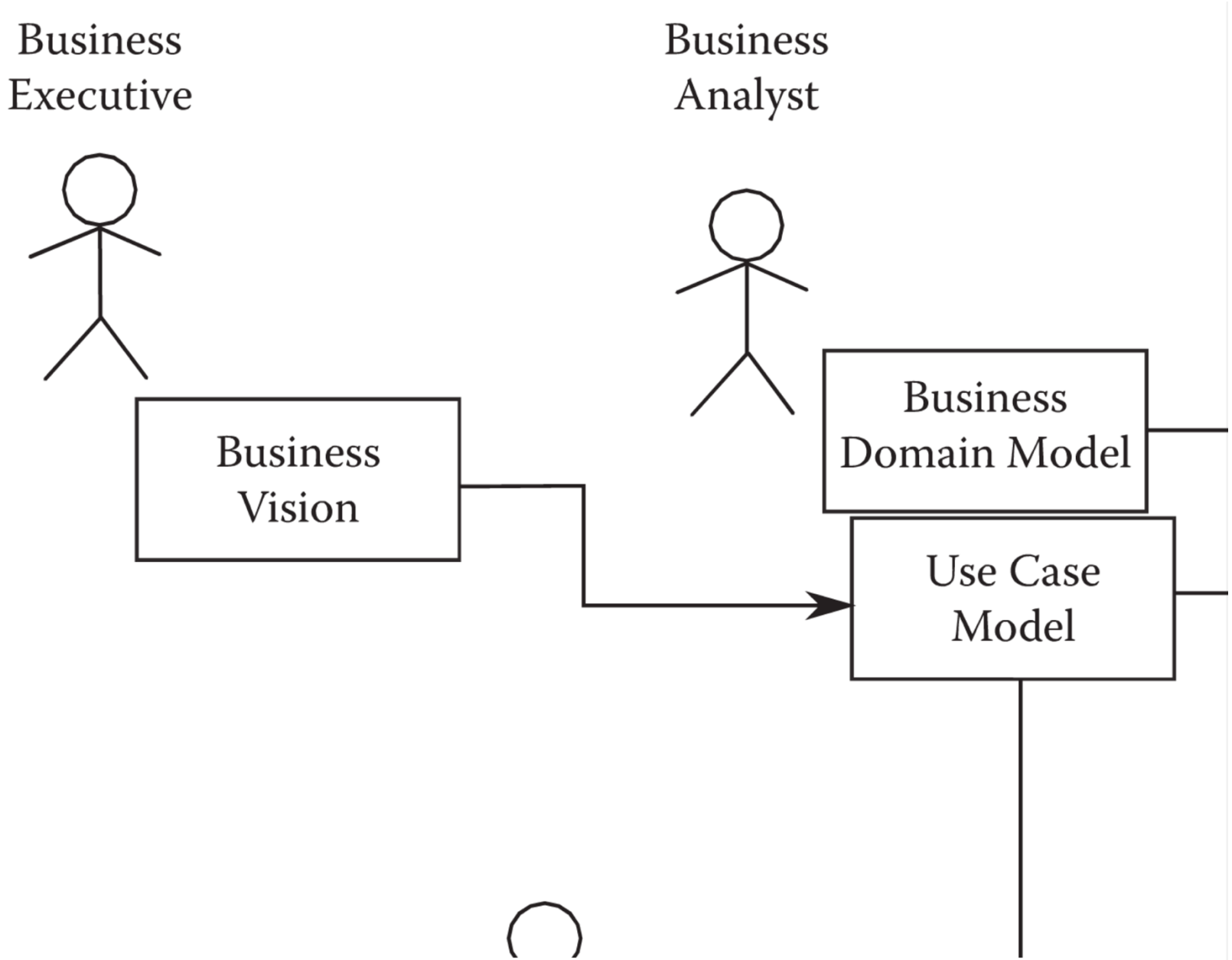
Multiple impls.

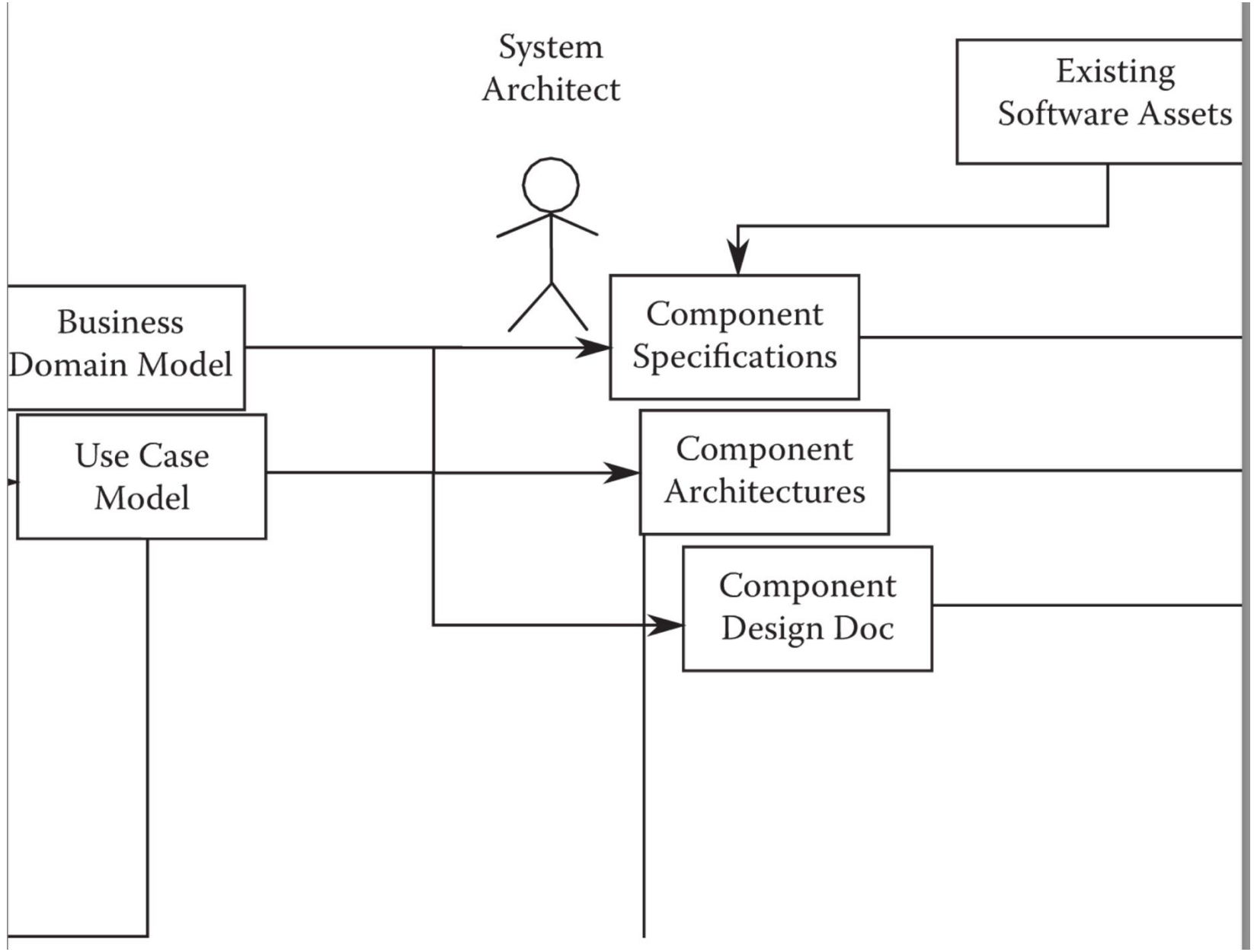
Process Stages

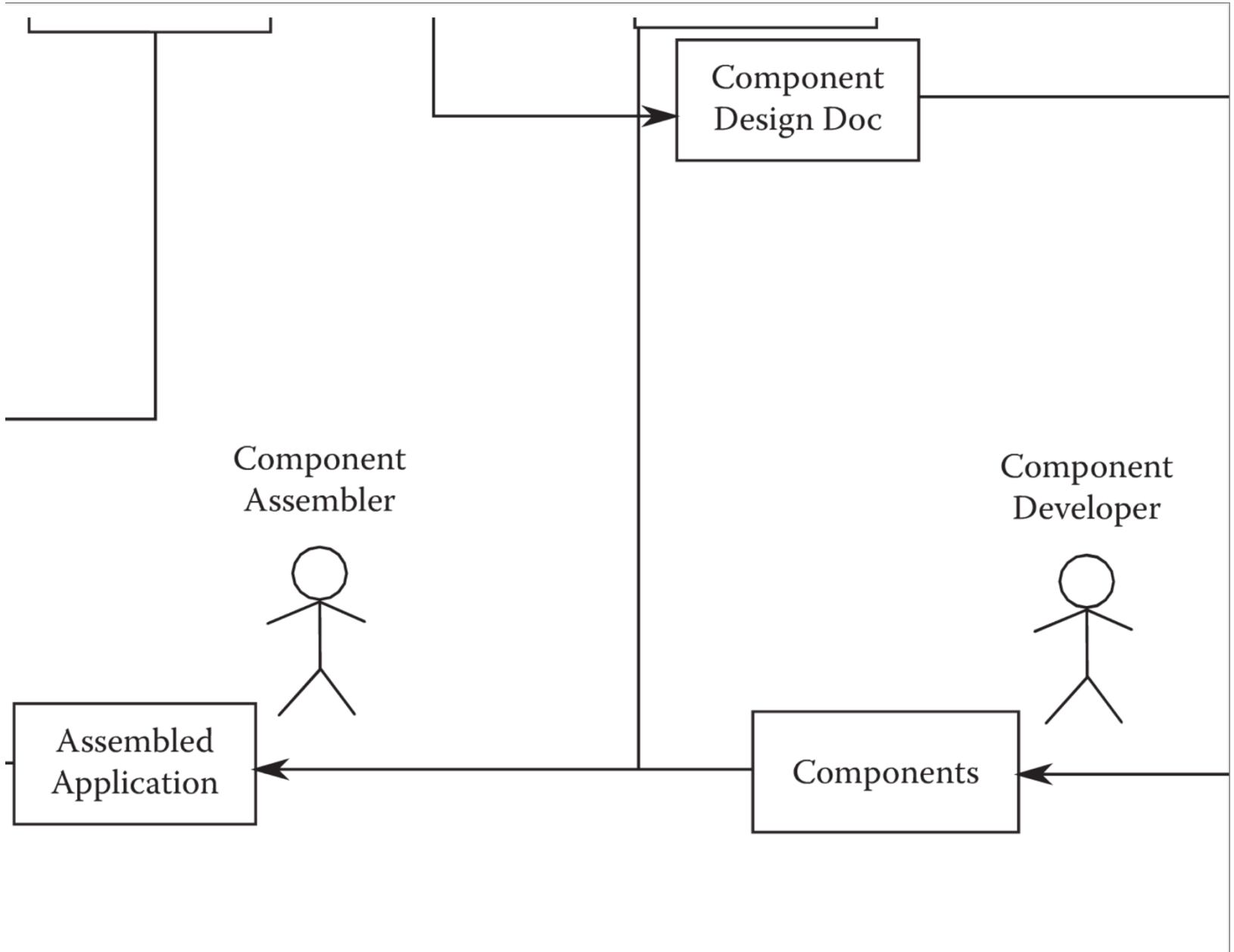


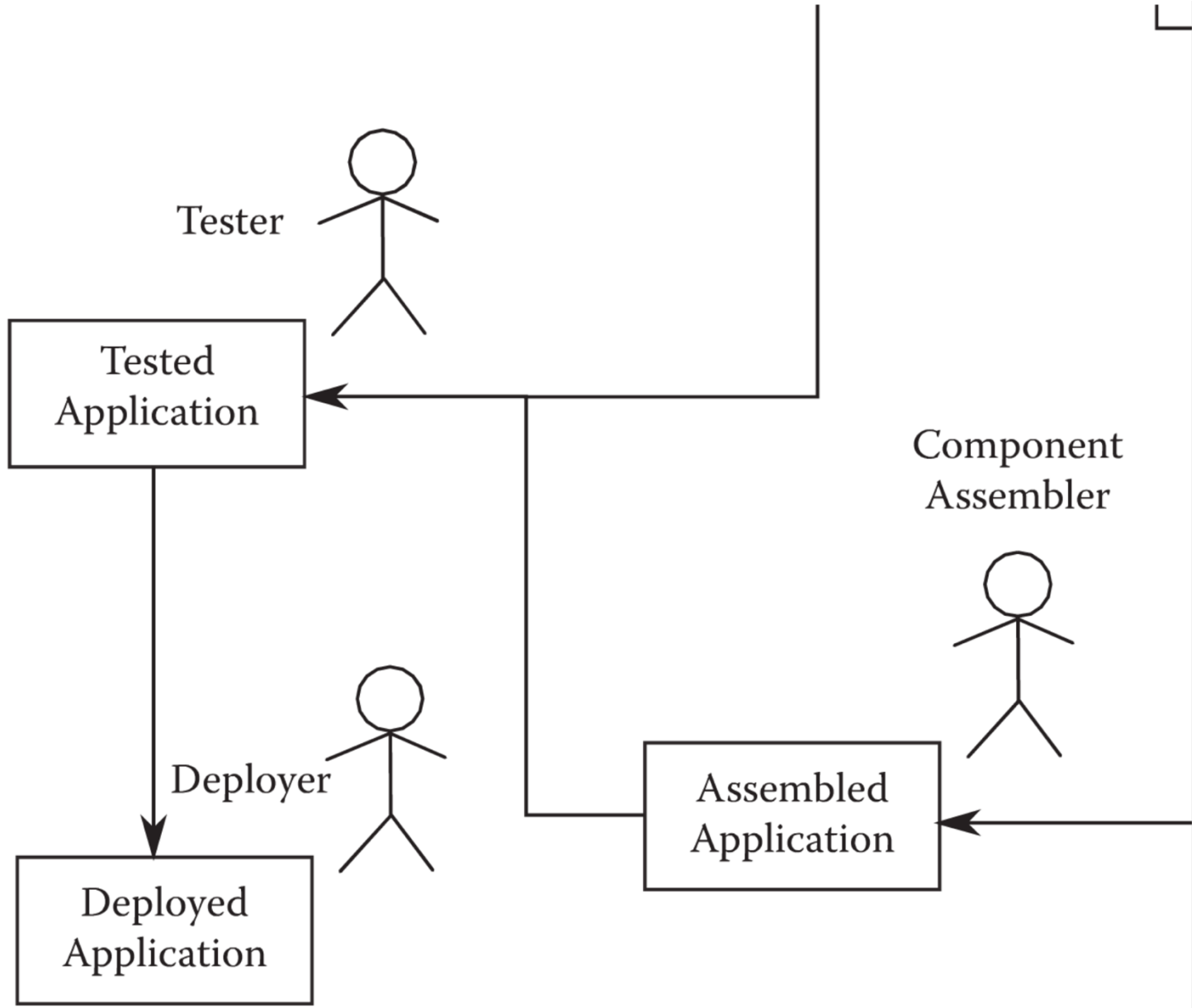
Life-Cycle


Business Executive
Business Analyst
System Architect
Existing Software Assets
Business Vision
Business Domain Model
Component Specifications
Use Case Model
Component Architectures
Component Design Doc
Tester
Tested Application
Component Assembler
Component Developer
Deployer
Assembled Application
Components
Deployed Application


Business
Executive
Business
Analyst
Business
Vision
Business
Domain Model
Use Case
Model


System
Architect
Existing
Software Assets
Business
Domain Model
Use Case
Model
Component
Specifications
Component
Architectures
Component
Design Doc


Component
Design Doc
Component
Assembler
Component
Developer
Assembled
Application
Components


Tester
Tested Application
Component Assembler
Deployer
Assembled Application
Deployed Application

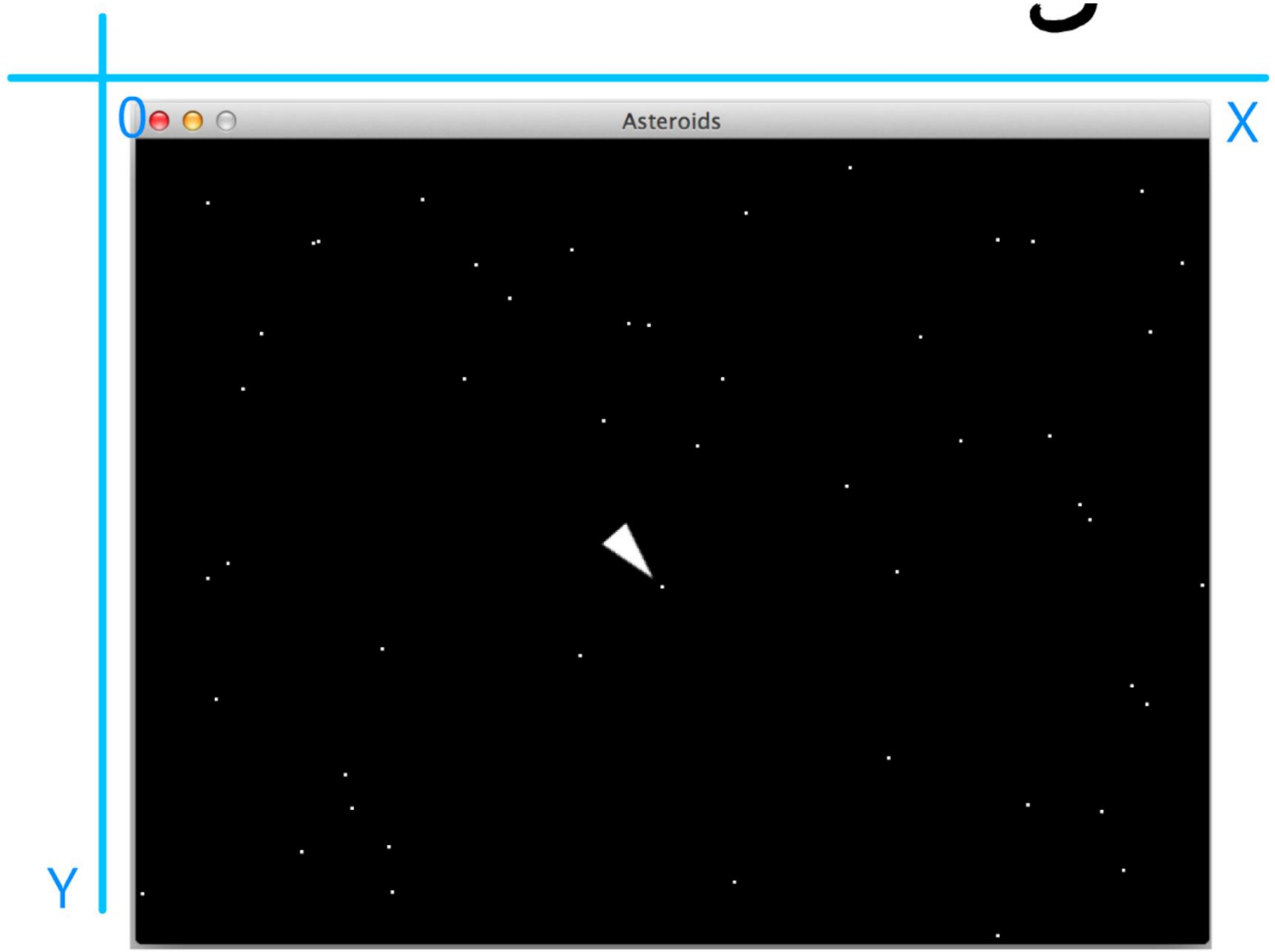
CBSE Advantages

- Reuse
- Contained Complexity
- Reduced cycle time
- Improved Quality
- Distributed development teams
- Reduced Maintenance cost

Summary

- A bit of history
- Component characteristics
- Software Components
- Benefits of CBSE

[IntroLab]



References

Math:

- Edwards, .C H., & Penney, D. .E (1997). Calculus with Analytic Geometry (5th Edition)
- Math dictionary - http://mathworld.wolfram.com